

Design Modeling: Design Class Diagram

Instructor: Muhammad Zeeshan Nazar

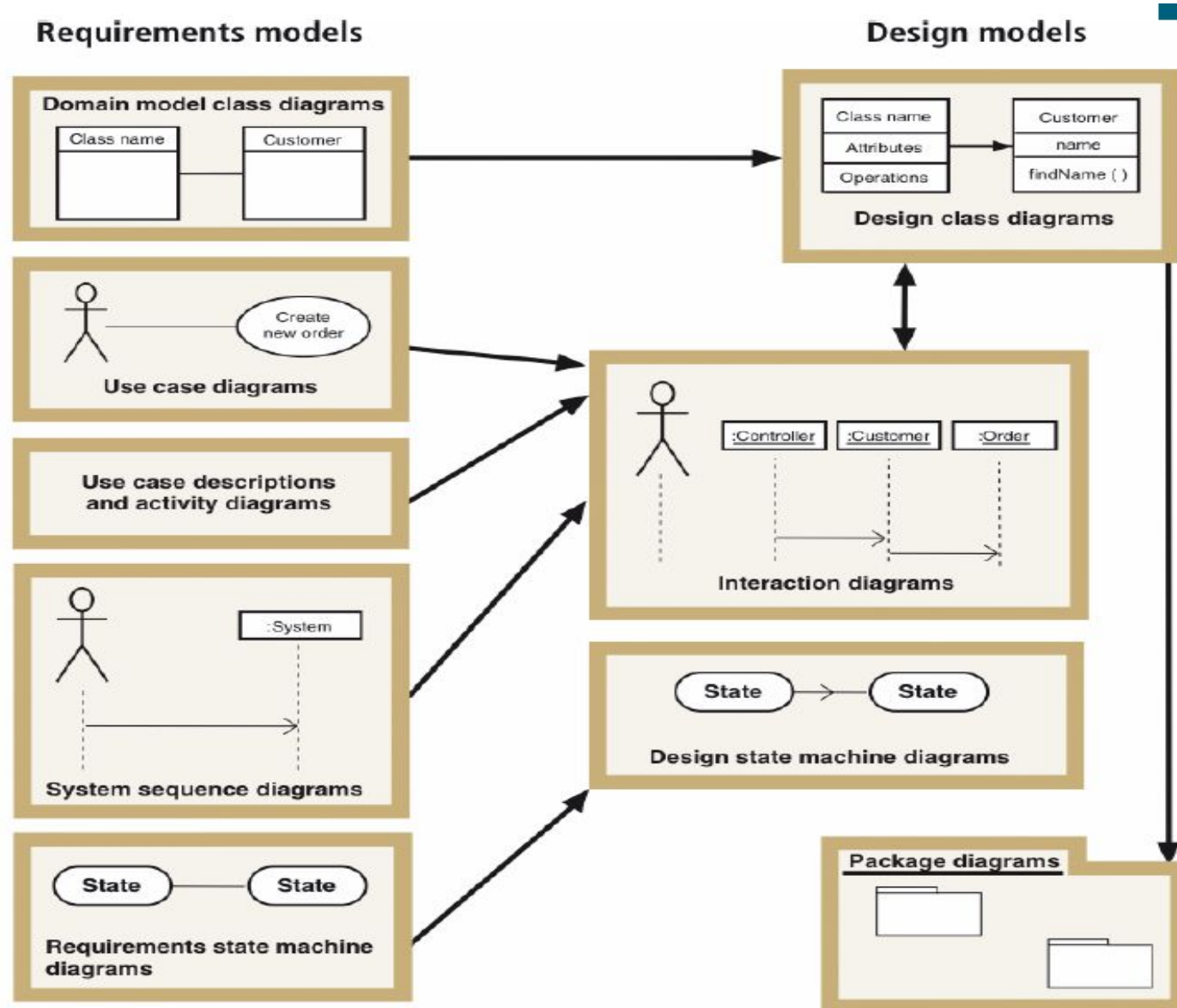
Object-Oriented Design—The Bridge Between Analysis and Programming

- Bridge between **users' requirements** and new system's **programming**
- Object-oriented design is process by which **detailed object-oriented models** are built
- Programmers use design to **write code** and **test** new system
- User interface, network, controls, security, and database require design tasks and models

Object-Oriented Design Processes and Models

- **Diagrams developed for analysis/requirements**
 - Use case diagrams and Use case descriptions
 - Activity diagrams
 - Domain model class diagrams
 - System sequence diagrams
- **Diagrams developed for design**
 - Interaction diagrams and package diagrams
 - Design class diagrams – include object-oriented classes, navigation between classes, attribute names, method names, and properties needed for programming

Design Models with Their Respective Input Models



Analysis Models

Design Models

Programming Models

Info about
things

Problem domain
class diagram

Design
class diagram

Info about
process flow

Use case
Descriptions

System Sequence
Diagrams

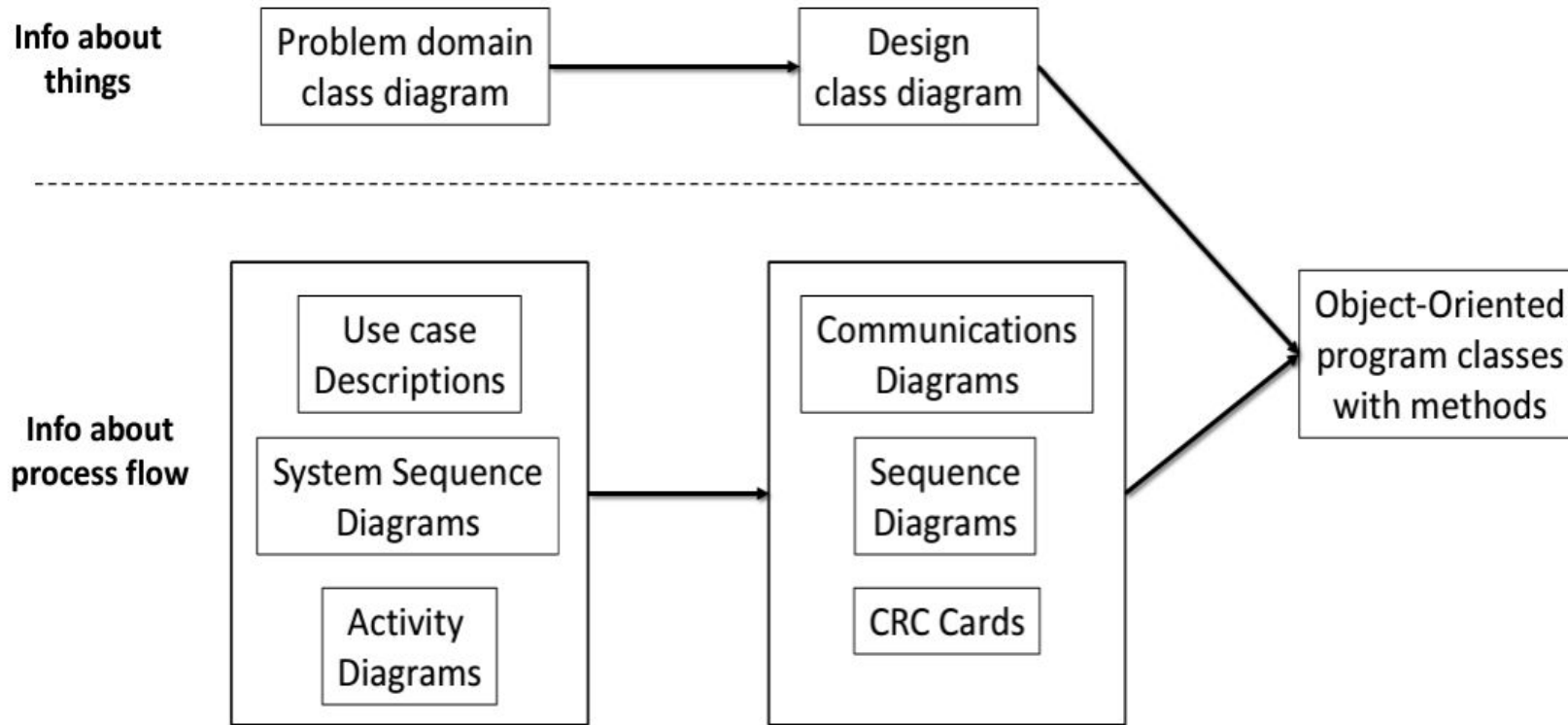
Activity
Diagrams

Communications
Diagrams

Sequence
Diagrams

CRC Cards

Object-Oriented
program classes
with methods



Iterative Process of OO Design - Design Steps

Overall design process

1. Develop the first-cut design class diagram showing navigation visibility.
2. Design each use case by developing a sequence diagram for each.
 - (a) Develop first-cut sequence diagrams.
 - (b) Develop multilayer sequence diagrams.
3. Update the design class by adding method signatures and navigation information.
4. Partition the solution into packages, as appropriate.

Design Classes, Interaction, and Design Process

- Design class diagrams and detailed interaction diagrams use each other as inputs and are developed in parallel
- First-cut design class diagram is based on **domain model** and **system design principles**
- First-cut sequence diagram for use case is extended from **system sequence diagram (SSD)**
 - Shows interacting objects
- Design class diagram is updated based on sequence diagram

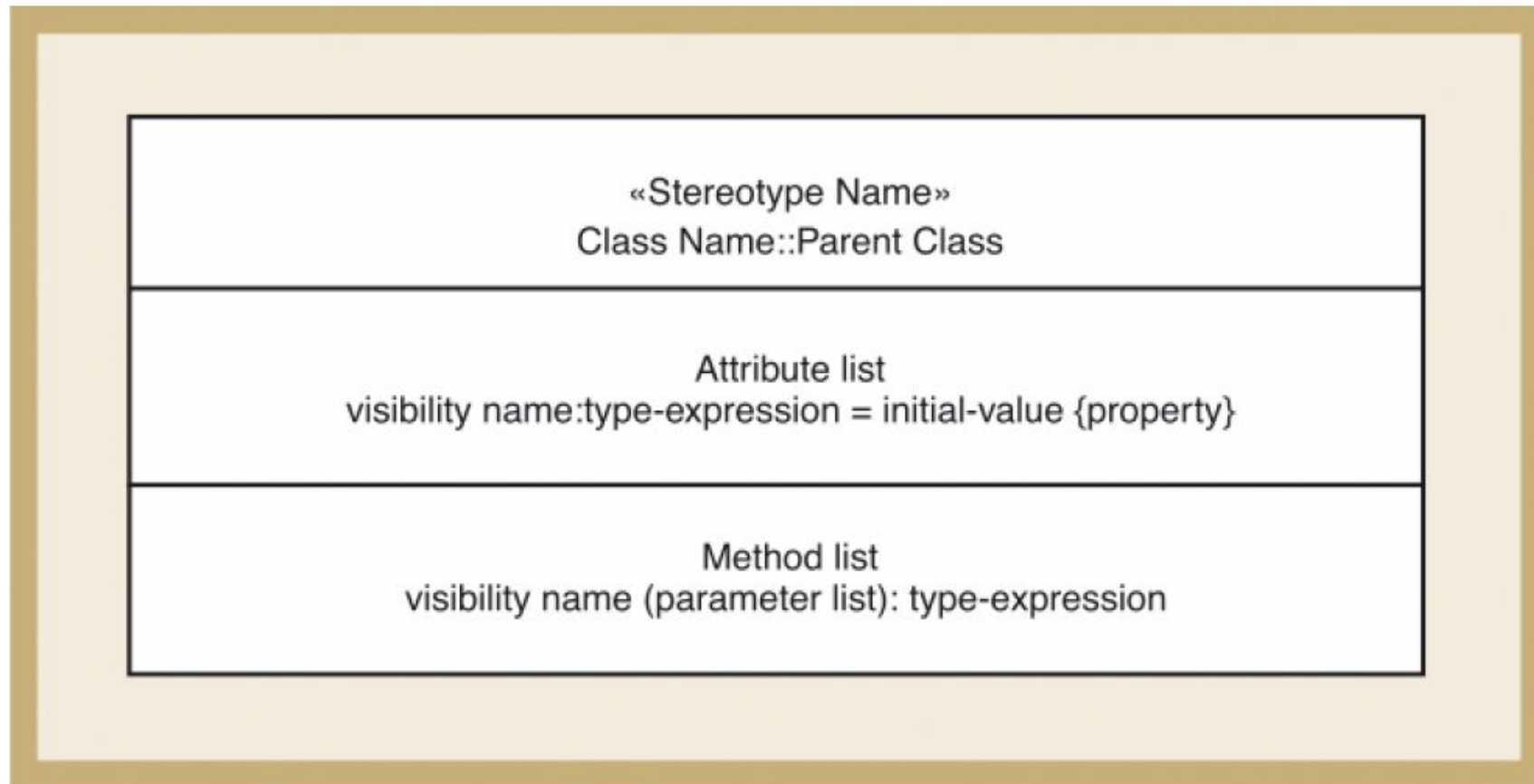
Design Class Notation

- **Name** – class name and stereotype information
- **Attribute visibility** (private or public) – attribute name, type expression, initial-value, property
- **Method signature** – information needed to invoke (or call) the method
 - Method visibility, method name, type-expression (return parameter), method parameter list (incoming arguments)
 - Overloaded method – method with same name but two or more different parameter lists
 - Class-level method – method associated with class instead of each object (static or shared method), denoted by an underline

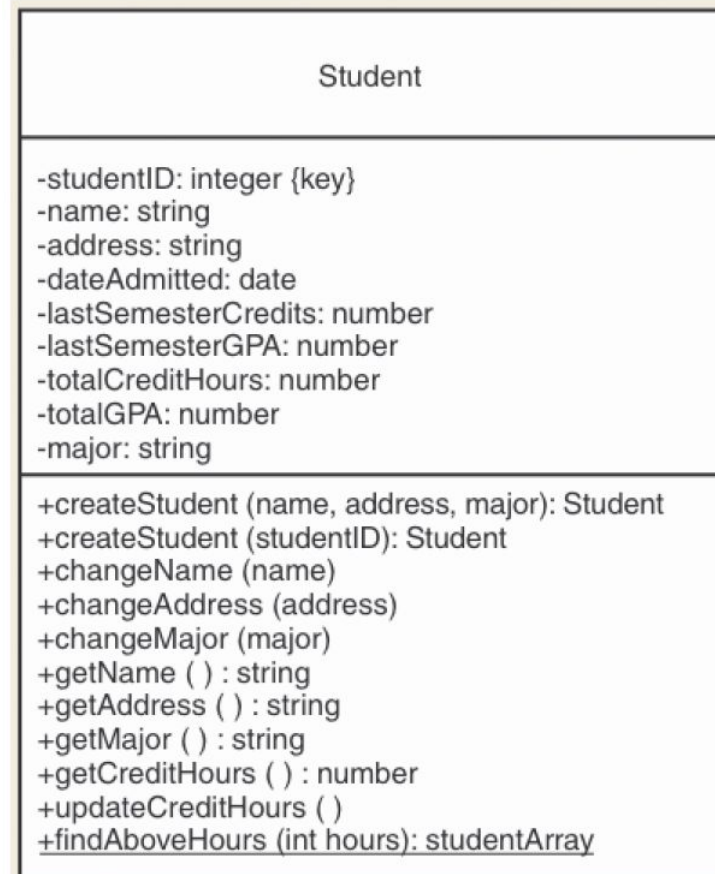
Visibility Symbols

<u>Notation</u>	<u>Visibility</u>
+	Public
-	Private
#	Protected
~	Package

Notation Used to Define a Design Class



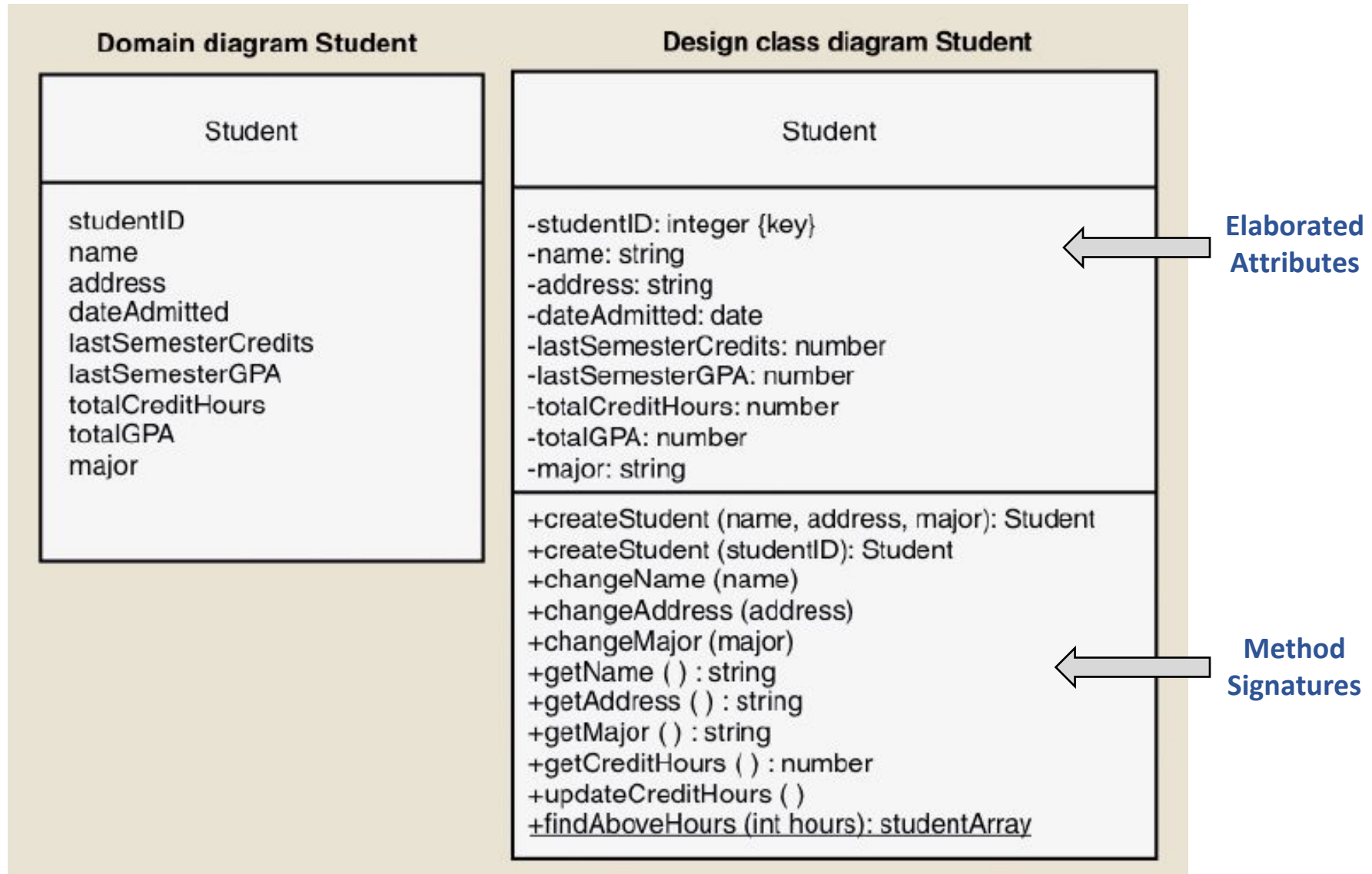
Student Design Class Example

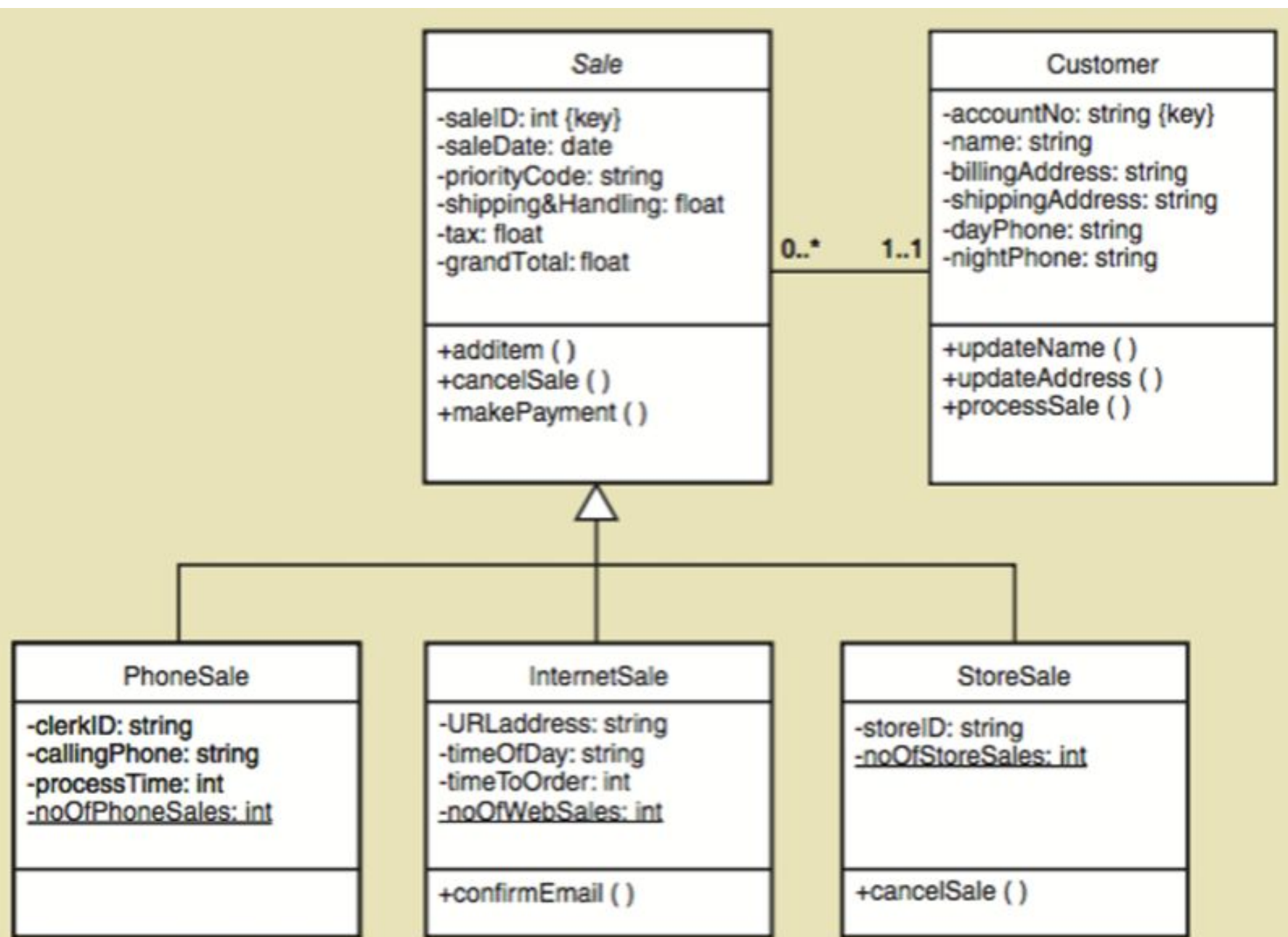


Creating the Design Class Diagram

- The following processes are done for each use case
- Start with Analysis Models
- In particular, the Domain Model Class Diagram
- Need to add:
 - Types and visibility of attributes
 - Methods, parameters, and return types
 - Classes outside the user's domain

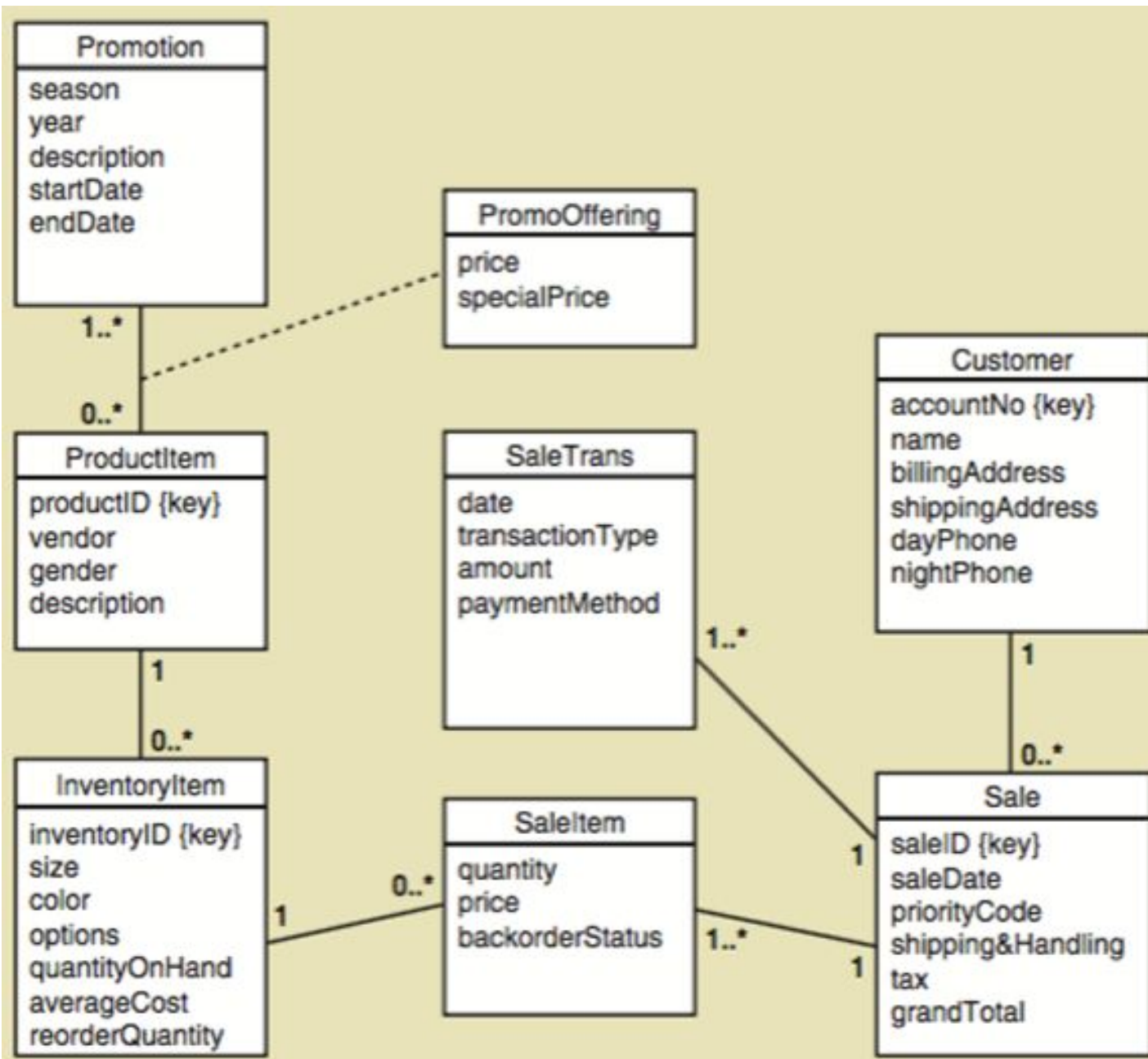
Student Class Examples for the Domain Class and the Design Class Diagrams

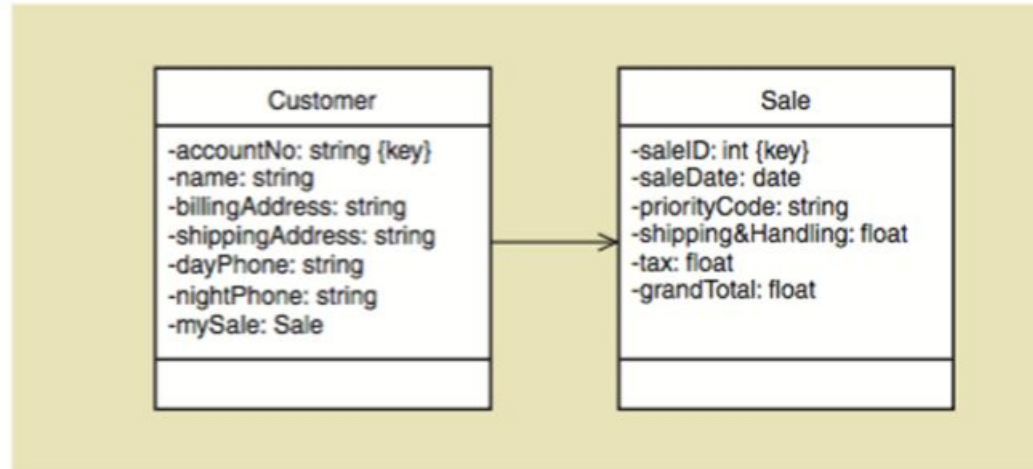
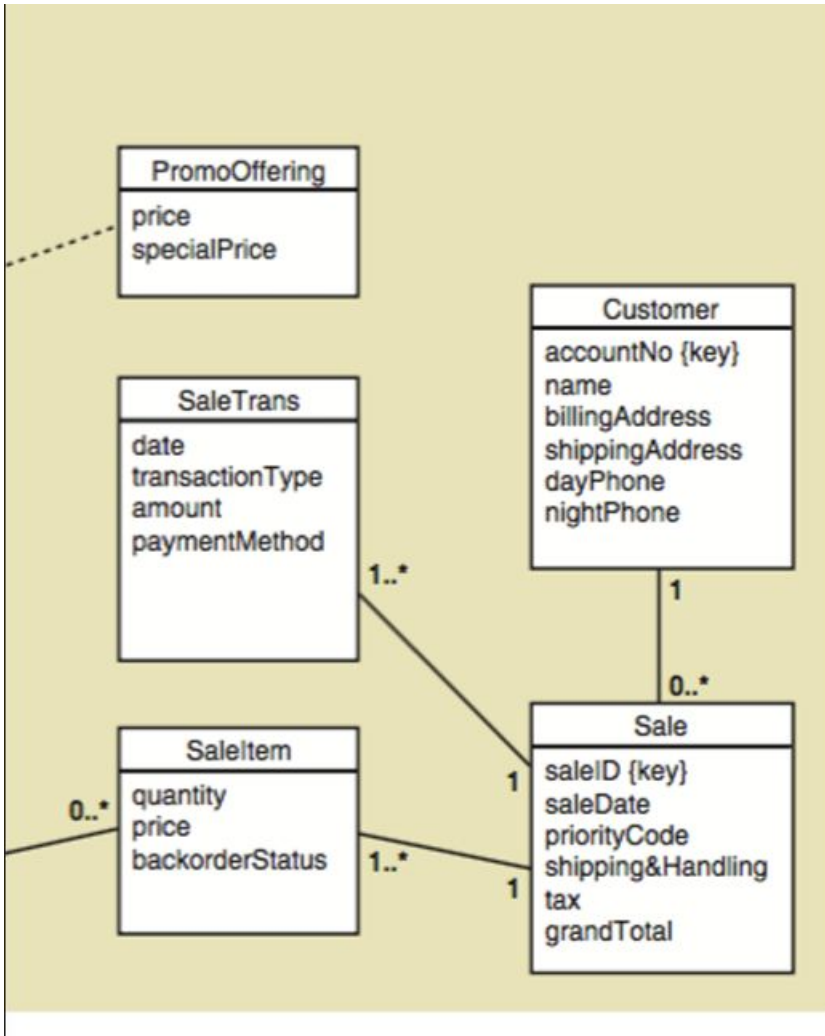




Developing First-Cut Design Class Diagram

1. Elaborate attributes
2. Navigation visibility
 - A design principle in which one object has a reference to another object and thus can interact with it
 - Can interact with other object by sending messages



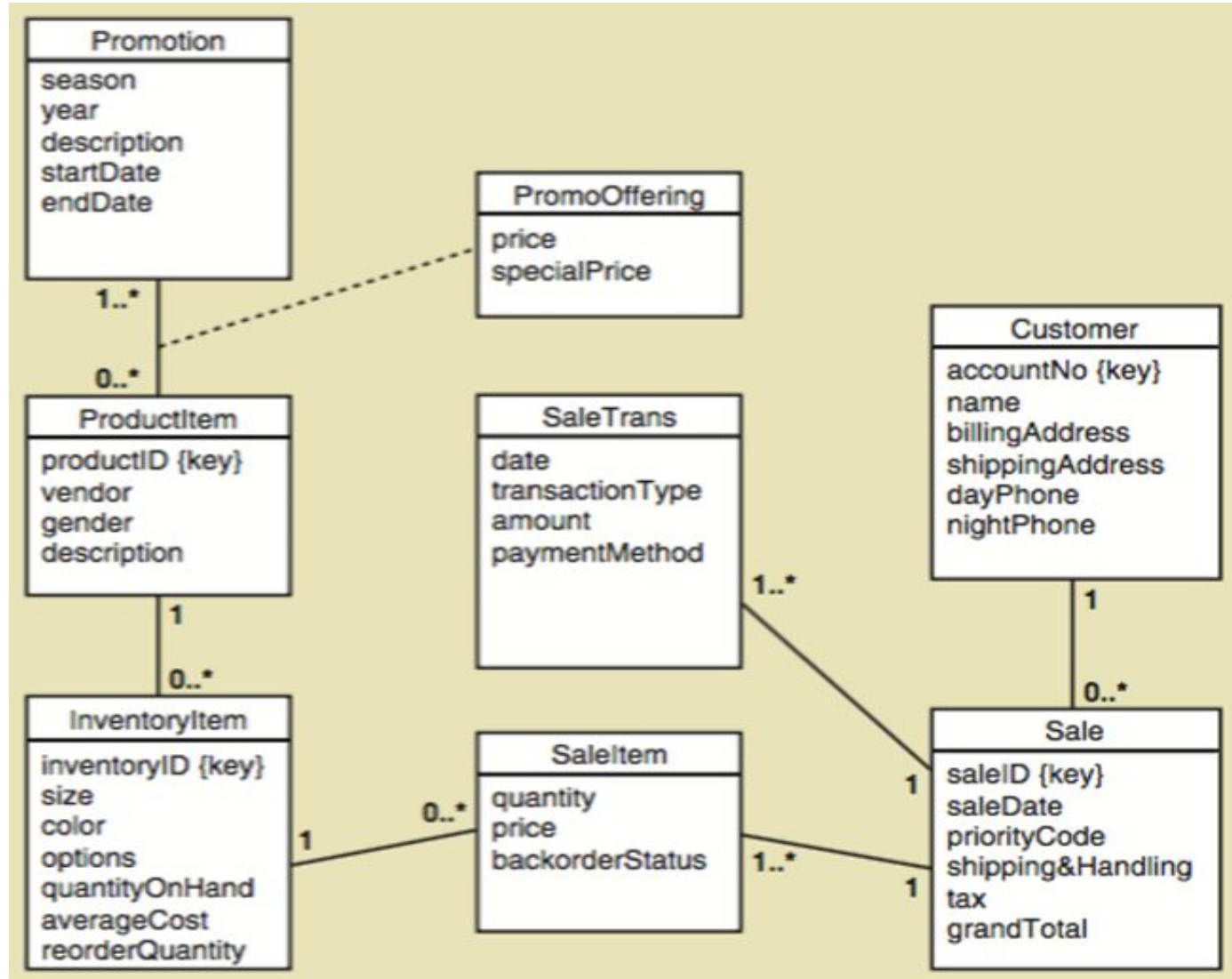


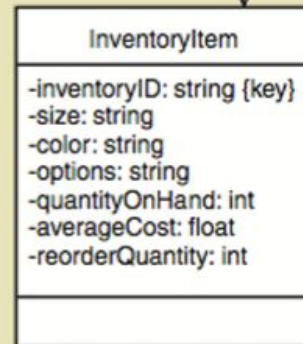
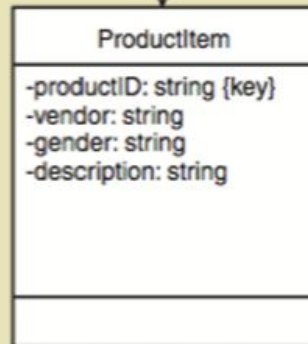
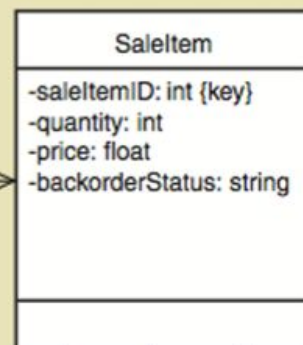
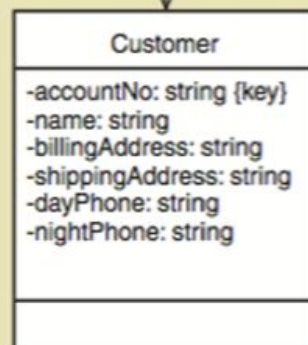
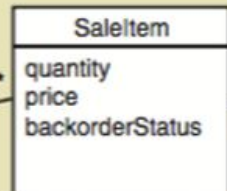
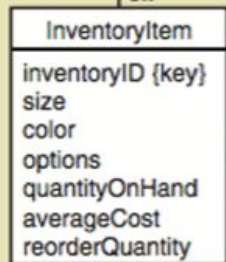
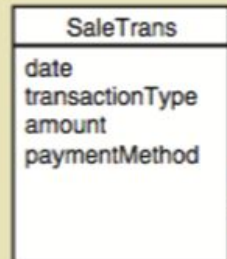
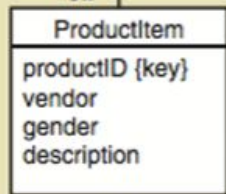
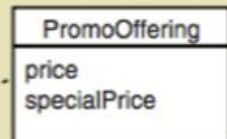
Adding Navigation Visibility

Identify:

- 1 to many relationships
 - If it is a superior/subordinate relationship, add visibility from superior to subordinate
 - Ex. From Sale to SaleItem
- For Mandatory associations where one can't live without the other
 - Add navigation from independent to dependent:
 - Ex. From Customer to Sale

Figure Out Where Navigation Visibility Must be Added

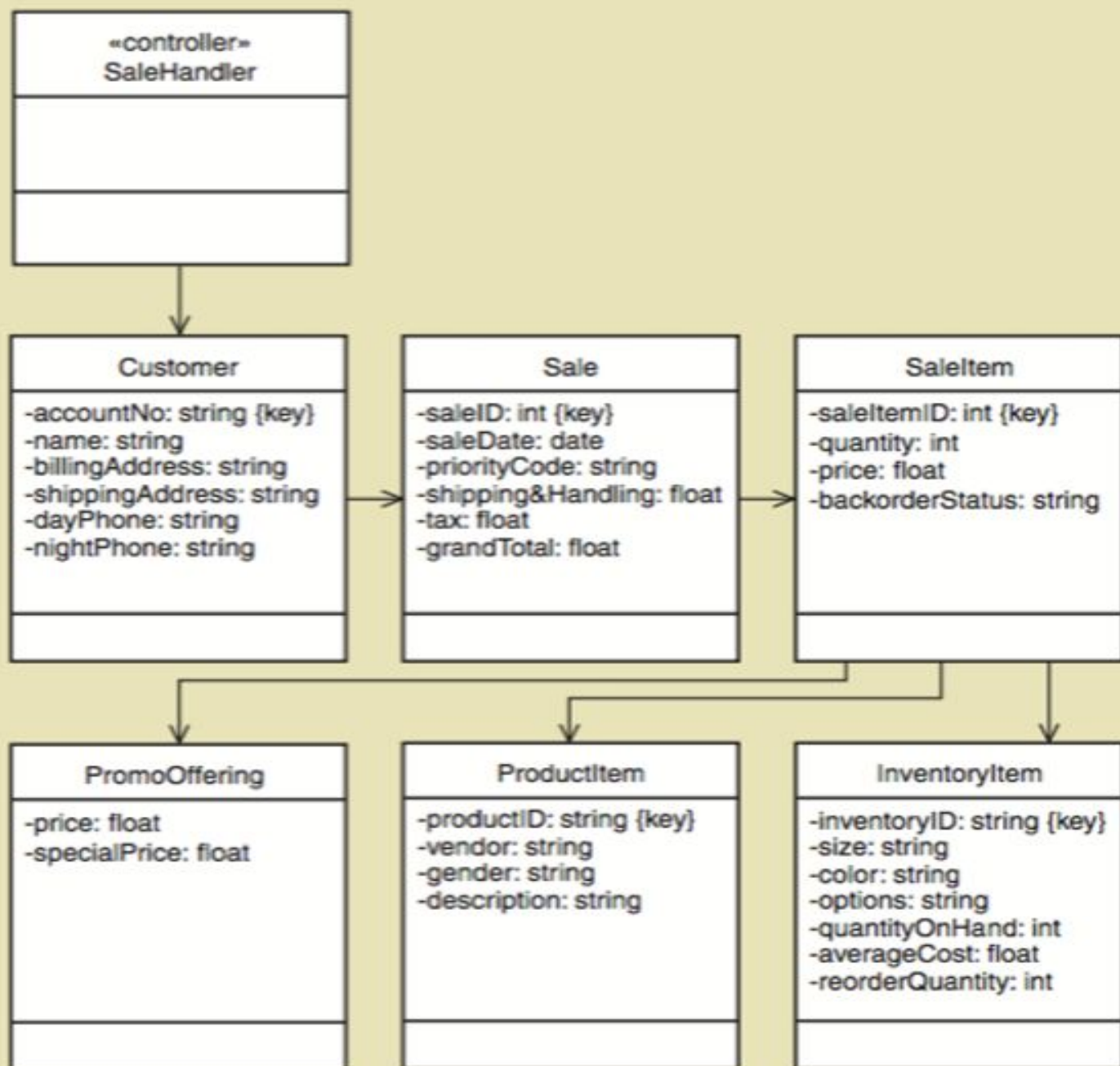




Building a First Cut Design Diagram

- Next step:

Add a controller class to act as a switch board between input screens and programming logic classes



Developing the First-Cut Design Class Diagram

- Extend domain model class diagram
 - Elaborate attributes with type and initial value information
- Detailed design proceeds use case-by-use case
 - Interaction diagrams implement navigation
 - Navigation arrows are updated to be consistent
 - Method signatures are added to each class

Developing the First-Cut Design Class Diagram

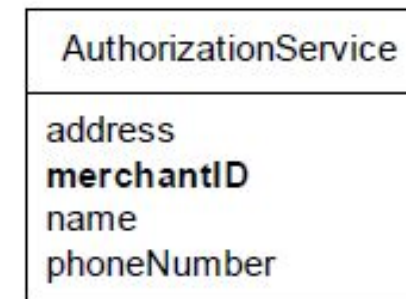
- Choose classes involved with the use case
- Add use case controller
- Elaborate attributes
 - Visibility, type-expression, initial-value, property
- Establish first-cut navigation visibility
 - One-to-many relationships usually navigated from superior to subordinate
 - Mandatory relationships usually navigated from independent to dependent
 - When an object needs information from another object, navigation arrow points to the object itself or to its parent in hierarchy
 - Navigation can be in both directions (arrows bidirectional)

Association Classes

- The following domain requirements set the stage for association classes:
 - Authorization services assign a merchant ID to each store for identification during communications.
 - A payment authorization request from the store to an authorization service needs the merchant ID that identifies the store to the service.
 - Furthermore, a store has a different merchant ID for each service.
- Where in the Domain Model should the merchant ID attribute reside?
- Placing *merchantID* in *Store* is incorrect because a *Store* can have more than one value for *merchantID*. The same is true with placing it in *Authorization-Service*

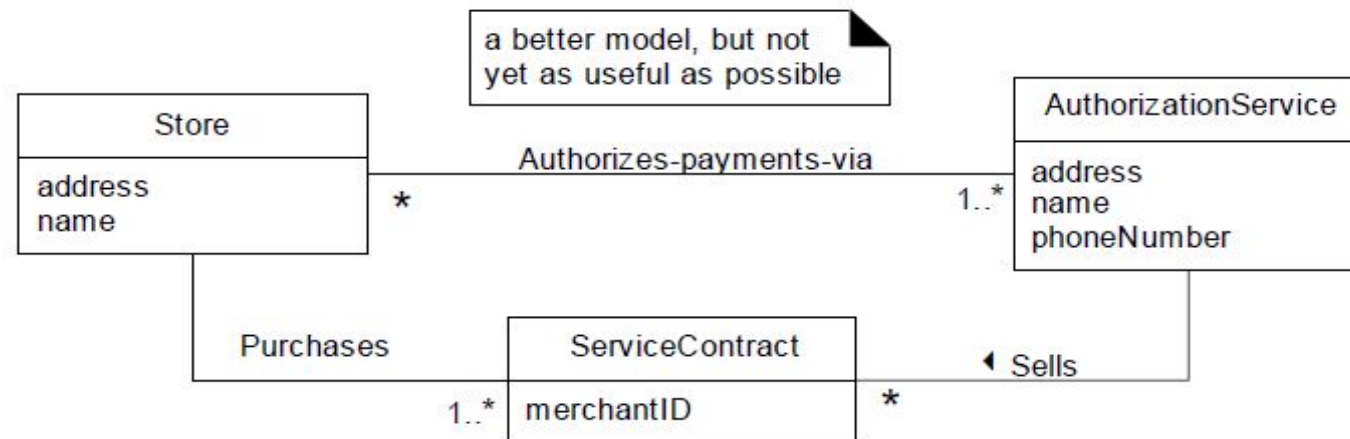


both placements of
merchantID are incorrect
because there may be more
than one merchantID



Association Classes

- This leads to the following modeling principle:
 - In a domain model, if a class C can simultaneously have many values for the same kind of attribute A, do not place attribute A in C. Place attribute A in another class that is associated with C.
- For example:
 - A *Person* may have many phone numbers. Place phone number in another class, such as *PhoneNumber* or *ContactInformation*, and associate many of these to *Person*.

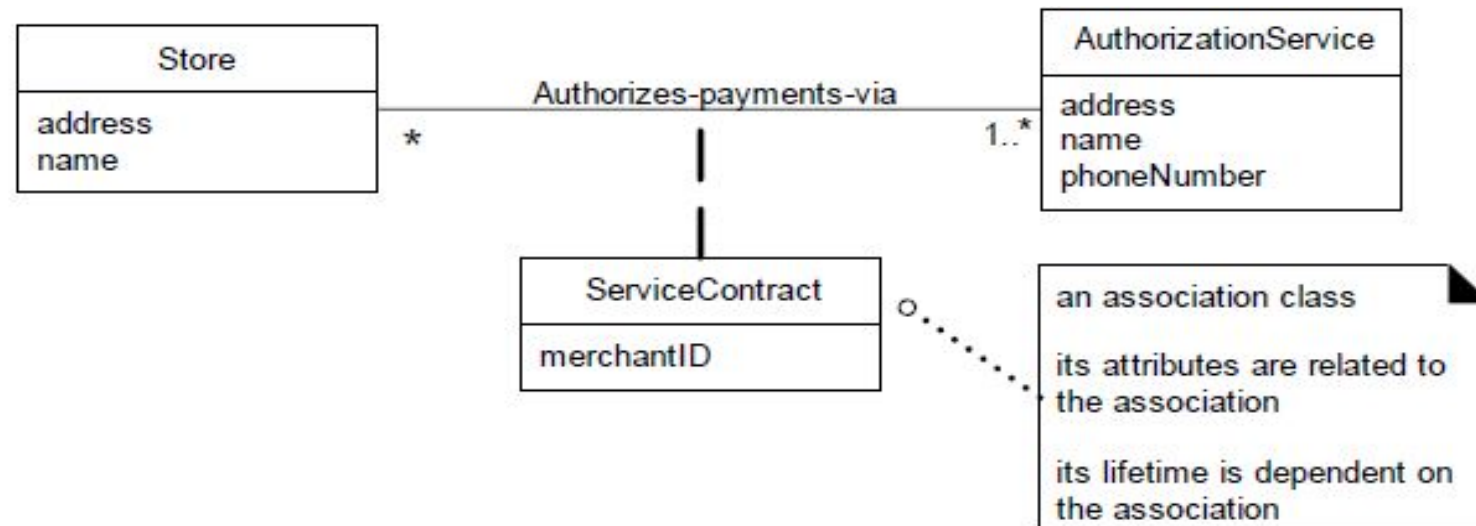


Association Classes

- In the business world, what concept formally records the information related to the services that a service provides to a customer?—a *Contract or Account*.
- The fact that both *Store* and *AuthorizationService* are related to *ServiceContract* is a clue that it is dependent on the relationship between the two.
- The *merchantID* may be thought of as an attribute related to the association between *Store* and *AuthorizationService*.
- This leads to the notion of an **association class**, in which we can add features to the association itself. *ServiceContract* may be modeled as an association class related to the association between *Store* and *AuthorizationService*.

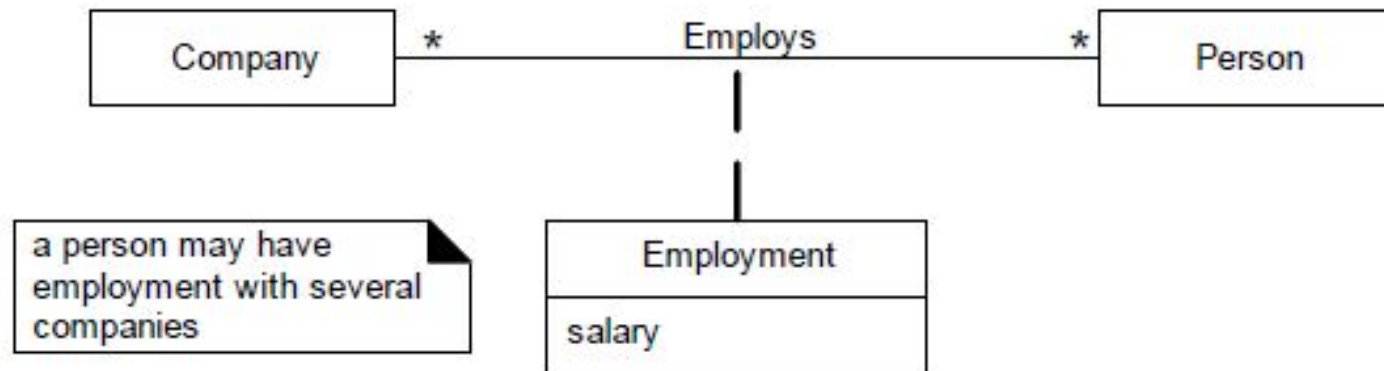
Association Classes

- In the UML, this is illustrated with a dashed line from the association to the association class.
- Figure visually communicates the idea that a *Service-Contract* and its attributes are related to the association between a *Store* and *AuthorizationService*, and that the lifetime of the *ServiceContract* is dependent on the relationship.

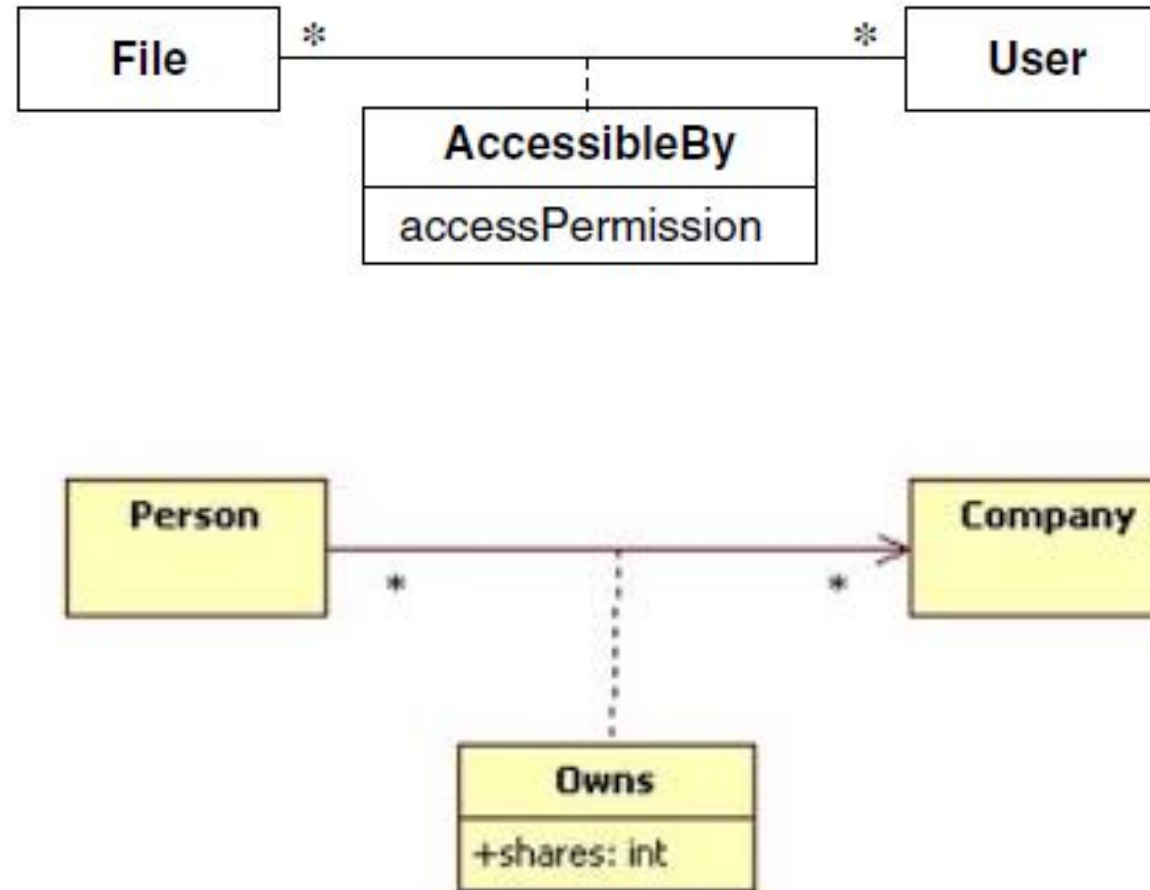


Association Classes

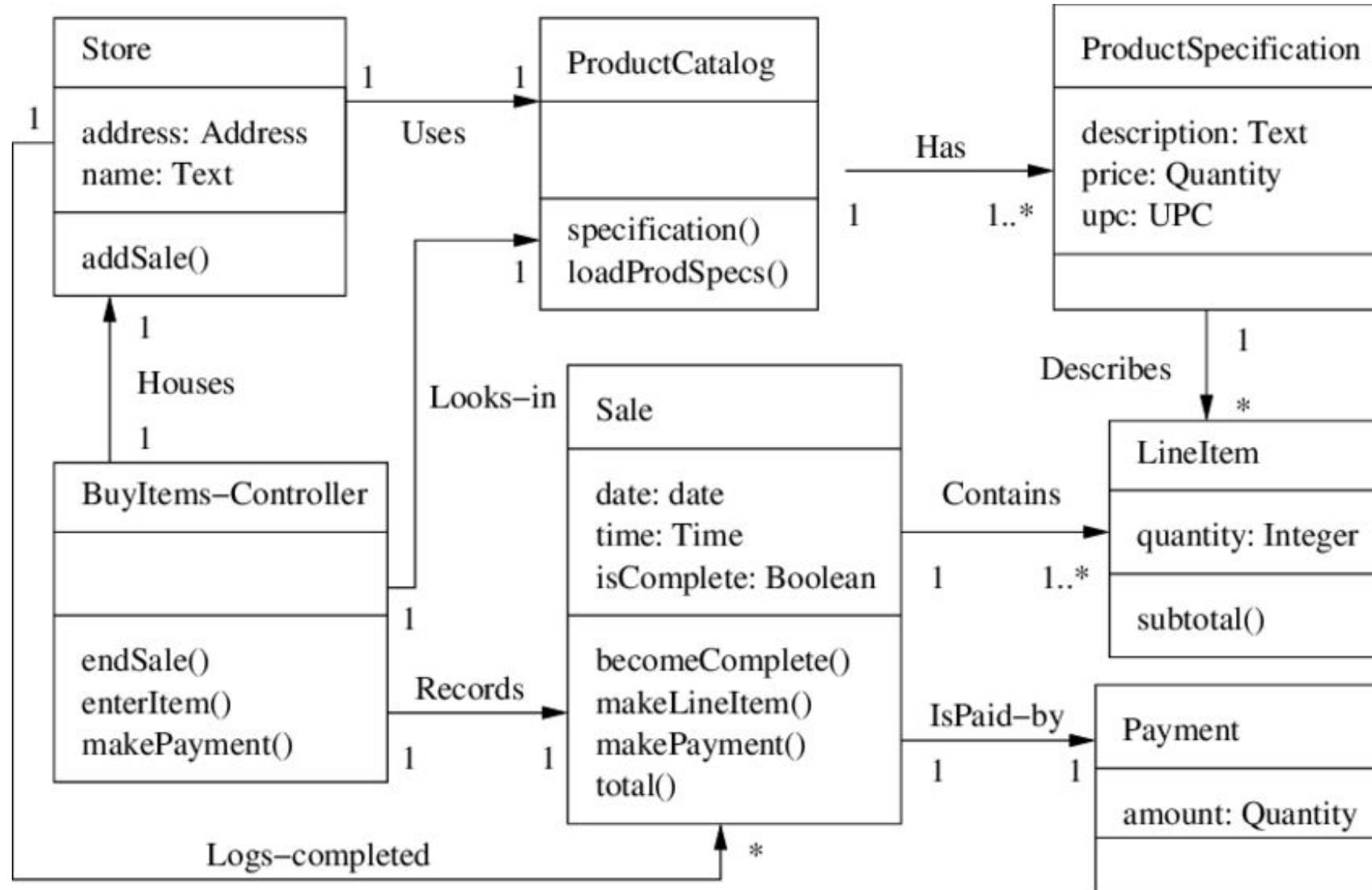
- Clues that an association class might be useful in a domain model:
 - An attribute is related to an association.
 - Instances of the association class have a life-time dependency on the association.
 - There is a many-to-many association between two concepts, and information associated with the association itself.



Association Classes



Design Class Diagram



References

- Satzinger, John W., Robert B. Jackson, and Stephen D. Burd. *Systems analysis and design in a changing world*. 4th Edition.
- Larman, Craig. *Applying UML and patterns: an introduction to object oriented analysis and design and iterative development*. Pearson Education India, 2012.
- *Object-Oriented Modeling and Design with UML*, Michael R. Blaha and James R. Rumbaugh, 2nd Edition, Pearson, 2005.